

Name: Muhammad Farhan
Roll no: 25K-0538
Course: Data Structures – Lab 3
Course Instructor: Miss Fatima Gado
Date of Submission: 02/09/2026

Task 1:

```
#include <iostream>
using namespace std;

int bubble_sort(int size, int arr[]);

int main()
{
    int size = 9;
    cout << "Enter the size of the array: ";
    cin >> size;

    int* arr = new int[size];
    cout << "Enter the elements of the array: \n";
    for (int i = 0; i < size; ++i)
    {
        cout << "Enter element #" << i + 1 << ": ";
        cin >> arr[i];
        cout << endl;
    }

    cout << "Initial Array: \n";
    for (int i = 0; i < size; ++i)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    int swaps = bubble_sort(size, arr);

    cout << "Sorted Array: \n";
    for (int i = 0; i < size; ++i)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "The Number of Swaps: " << swaps << endl;

    return 0;
}
```

```

int bubble_sort(int size, int arr[])
{
    int temp, counta=0;
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size - 1; j++)
        {
            if (arr[j] < arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                counta++;
            }
        }
    }

    return counta;
}

```

```

Enter the size of the array: 5
Enter the elements of the array:
Enter element #1: 3

Enter element #2: 8

Enter element #3: 5

Enter element #4: 7

Enter element #5: 1

Initial Array:
3 8 5 7 1
Sorted Array:
8 7 5 3 1
The Number of Swaps: 4

```

Task 2:

```
#include <iostream>
using namespace std;

void bubble_sort(int size, int arr[],int &comparisions,int &swaps);

int main()
{
    int size = 7;
    int arr[7] = { 1200, 450, 800, 450, 1500, 700, 1200 };

    cout << "Initial Array: \n";
    for (int i = 0; i < size; ++i)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    int comparisions = 0, swaps = 0;
    bubble_sort(size, arr, comparisions, swaps);

    cout << "Sorted Array: \n";
    for (int i = 0; i < size; ++i)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "The Number of Swaps: " << swaps << endl;
    cout << "The Number of Comparisions: " << comparisions << endl;
    return 0;
}

void bubble_sort(int size, int arr[], int &comparisions, int &swaps)
{
    int temp;
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size - 1; j++)
        {
            comparisions++;
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swaps++;
            }
        }
    }
    cout << "Price: " << arr[size - 1 - i] << " has reached its final position." << endl;
}
```

}

```
Initial Array:
1200 450 800 450 1500 700 1200
Price: 1500 has reached its final position.
Price: 1200 has reached its final position.
Price: 1200 has reached its final position.
Price: 800 has reached its final position.
Price: 700 has reached its final position.
Price: 450 has reached its final position.
Price: 450 has reached its final position.
Sorted Array:
450 450 700 800 1200 1200 1500
The Number of Swaps: 8
The Number of Comparisions: 42
```

Task 3:

```
#include <iostream>
using namespace std;

void insertion_sort(int size, int arr[]);
int binary_search(int array[], int size, int val);

int main()
{
    int size = 7;
    int array[] = {78, 45, 92, 67, 55, 88, 73};

    //first, sort according to task.
    insertion_sort(size, array);

    cout << "Enter the K'th smallest score you want to find: ";
    int posi;
    cin >> posi;

    cout << "The " << posi << "th smallest score is: " << array[posi-1] << endl;

    /*
    now...the 4th and 5th requirement doesn't really make much sense.
    we just printed the kth smallest value from the array, so of course it exists in the array,
    so why do we need to binary research to see if it exists or not?
    therefore i will be changing it up to ask the user for any score,
    then we will do a binary search to see if that value is in the array or not.
    */

    cout << "Enter the score to see if it exists in the array: ";
    int score;
    cin >> score;

    int result = binary_search(array, size, score);

    if (result == -1)
    {
        cout << "\n\nThe input score was not in the array.";
    }
    else
    {
        cout << "\n\nThe input score was found at index: " << result << endl;
        cout << "Array[" << result << "] = " << array[result] << endl;
    }

    return 0;
}

void insertion_sort(int size, int arr[])
{
    int temp;
    for (int i = 1; i < size; ++i)
```

```

{
    temp = arr[i];
    int j;
    for (j = i; j > 0 && arr[j-1] > temp; j--)
    {
        arr[j] = arr[j-1];
    }
    arr[j] = temp;
}
}

```

```

int binary_search(int array[], int size, int val)
{
    int middle;
    int start = 0;
    int end = size-1;
    while(start <= end)
    {
        middle = (start + end)/2;
        if (array[middle] == val)
        {
            return middle;
        }
        if (array[middle] < val)
        {
            start = middle + 1;
        }
        if (array[middle] > val)
        {
            end = middle - 1;
        }
    }
    return -1;
}

```

```

Enter the K'th smallest score you want to find: 3
The 3'th smallest score is: 67
Enter the score to see if it exists in the array: 88

The input score was found at index: 5
Array[5] = 88

```

Task 4:

```
#include <iostream>
using namespace std;

void insertion_sort(int size, int arr[]);
int binary_search(int array[], int size, int val);

int main()
{
    int size = 7;
    int books[] = {105, 203, 150, 87, 310, 120, 250};

    cout << "Enter the Book ID you want to search: ";
    int ID;
    cin >> ID;

    //initial search.
    int result = binary_search(books, size, ID);

    if (result == -1)
    {
        cout << "\n\nThe input ID could not be found.";
    }
    else
    {
        cout << "\n\nThe input ID was found at index: " << result << endl;
        cout << "Book[" << result << "] = " << books[result] << endl;
    }

    //now sort.
    insertion_sort(size, books);

    //again, search.
    result = binary_search(books, size, ID);

    if (result == -1)
    {
        cout << "\n\nThe input ID could not be found.";
    }
    else
    {
        cout << "\n\nThe input ID was found at index: " << result << endl;
        cout << "Book[" << result << "] = " << books[result] << endl;
    }

    return 0;
}

void insertion_sort(int size, int arr[])
{
    int temp;
    for (int i = 1; i < size; ++i)
```

```

{
    temp = arr[i];
    int j;
    for (j = i; j > 0 && arr[j-1] > temp; j--)
    {
        arr[j] = arr[j-1];
    }
    arr[j] = temp;
}
}

```

```

int binary_search(int array[], int size, int val)
{
    int middle;
    int start = 0;
    int end = size-1;
    while(start <= end) //if start becomes equal or greater than end, then that means we have
    completed our search.
    {
        middle = (start + end)/2;
        if (array[middle] == val)
        {
            return middle;
        }
        if (array[middle] < val)
        {
            start = middle + 1;
        }
        if (array[middle] > val)
        {
            end = middle - 1;
        }
    }
    return -1;
}

```

```

Enter the Book ID you want to search: 150

The input ID could not be found.

The input ID was found at index: 3
Book[3] = 150

```


Task 5:

```
#include <iostream>
using namespace std;

void selection_sort(int size, int arr[]);

int main()
{
    int size = 7;
    int salaries[] = {45000, 62000, 38000, 75000, 51000, 68000, 42000};

    selection_sort(size, salaries);

    cout << "Lowest Salary: " << salaries[0] << endl;
    if (size % 2)
    {
        cout << "Middle Salary: " << salaries[size/2] << endl;
    }
    else
    {
        cout << "Middle Salary: " << salaries[size/2 - 1] << " and " << salaries[size/2] << endl;
    }
    cout << "Highest Salary: " << salaries[size - 1] << endl;

    return 0;
}

void selection_sort(int size, int arr[])
{
    int temp, min, index;
    for (int i = 0; i < size-1; i++) //size-1 because the last element would be sorted by the 2nd last
loop anyway.
    {
        min = arr[i];
        index = i;
        for (int j = i+1; j < size; j++) //i+1 to avoid comparing arr[i] to itself o-n the first loop
        {
            if (arr[j] < min)
            {
                min = arr[j];
                index = j;
            }
        }
        if (arr[i] == min)
        {
            continue;
        }
        temp = arr[i];
        arr[i] = min;
        arr[index] = temp;
    }
}
```

Lowest Salary: 38000
Middle Salary: 51000
Highest Salary: 75000